

Contexto, Objetivo e Hipóteses do Projeto

- Contexto: . A obesidade é uma condição multifatorial complexa, influenciada por uma interação dinâmica entre genética, comportamento alimentar, níveis de atividade física e fatores socioeconômicos. Abordagens tradicionais de saúde frequentemente categorizam indivíduos baseando-se estritamente no Índice de Massa Corporal (IMC). No entanto, essa métrica unidimensional falha em capturar as nuances comportamentais que levam ao ganho de peso ou à manutenção de um estilo de vida saudável. Em um cenário de Health Analytics, tratar todos os indivíduos com "Sobrepeso" da mesma forma é ineficiente. Um indivíduo jovem que se alimenta mal por falta de tempo requer uma intervenção completamente diferente de um indivíduo sênior sedentário por limitações de mobilidade. Portanto, a aplicação de técnicas de Aprendizado de Máquina Não Supervisionado tornam-se essenciais para transitar de uma segmentação puramente clínica para uma segmentação comportamental acionável. - Objetivo: O objetivo central deste projeto é identificar e caracterizar agrupamentos naturais (clusters) de indivíduos dentro do dataset de níveis de obesidade da UCI, utilizando dados demográficos, hábitos alimentares e condições físicas. - Os objetivos específicos incluem: . Mapear Padrões Latentes: Descobrir associações não-lineares entre variáveis (ex: relação entre uso de tecnologia, transporte e peso) que não são óbvias em análises univariadas; . Segmentação de Perfis (Profiling): Criar "personas" baseadas em dados que descrevam grupos distintos de comportamento, indo além dos rótulos de classe tradicionais; . Validação de Separabilidade: Investigar, através de redução de dimensionalidade (PCA, t-SNE, UMAP), se os diferentes níveis de obesidade possuem fronteiras biológicas rígidas ou se comportam como um espectro contínuo. - Hipóteses: . H1 – Heterogeneidade dentro do mesmo IMC: Indivíduos com o mesmo nível de IMC podem pertencer a clusters distintos devido a hábitos de vida opostos (ex: "Falso Magro" sedentário vs. "Atleta" com alta massa muscular); . H2 – O Espectro da Obesidade: A transição entre peso normal, sobrepeso não apresenta fronteiras rígidas, formando um continuum comportamental difícil de separar linearmente; . H3 – A Obesidade Mórbida como Outlier: Espera-se que os casos de Obesidade Tipo III apresentem um padrão fenotípico e comportamental radicalmente distinto, isolando-se geometricamente dos demais grupos; . H4 – Influência Geracional: A idade e o meio de transporte atuarão como variáveis discriminantes fortes, separando perfis baseados em estágio de vida (Estudantes vs. Profissionais Sêniores).

Carregamento de Dados e Entendimento Inicial dos Dados

- Esta seção apresenta o carregamento do dataset e uma verificação inicial de sua estrutura, incluindo dimensionalidade, tipos de variáveis e possíveis problemas evidentes, como valores faltantes ou inconsistências. - Este conjunto de dados inclui informações para a estimativa dos níveis de obesidade em indivíduos do México, Peru e Colômbia, com base em seus hábitos alimentares e condição física. Os dados contêm 17 atributos e 2111 registros, os quais são rotulados com a variável de classe NObesity (Nível de Obesidade), que permite a classificação dos dados utilizando os valores de Peso Insuficiente, Peso Normal, Sobrepeso Nível I, Sobrepeso Nível II, Obesidade Tipo I, Obesidade Tipo II e Obesidade Tipo III. 77% dos dados foram gerados sinteticamente utilizando a ferramenta Weka e o filtro SMOTE, e 23% dos dados foram coletados diretamente dos usuários por meio de uma plataforma web.

- Carregamento das bibliotecas e dados

```
In [47]: # =====  
# 1. Bibliotecas padrão  
# =====  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
import warnings  
warnings.filterwarnings("ignore")  
  
# =====  
# 2. Bibliotecas científicas / estatísticas  
# =====  
from scipy import stats
```

```

from scipy.stats import chi2_contingency
from scipy.stats import f_oneway

# =====
# 3. Redução de dimensionalidade e visualização avançada
# =====
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
import umap.umap_ as umap

# =====
# 4. Pré-processamento
# =====
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

# =====
# 5. Algoritmos de clustering
# =====
import scipy.cluster.hierarchy as sch
from sklearn.cluster import KMeans, AgglomerativeClustering, Birch
from sklearn.mixture import GaussianMixture
from sklearn.metrics import (
    silhouette_score,
    davies_bouldin_score,
    calinski_harabasz_score
)

# =====
# 6. Outras utilidades
# =====
import joblib
from ucimlrepo import fetch_ucirepo

```

```

In [2]: # Dataset
obesity = fetch_ucirepo(id=544)

X = obesity.data.features
y = obesity.data.targets

# Combinar tudo em um único dataframe
df = pd.concat([X, y], axis=1)

df.head()

```

Out[2]:	Gender	Age	Height	Weight	family_history_with_overweight	FAVC	FCVC	NCP		
0	Female	21.0	1.62	64.0		yes	no	2.0	3.0	Some
1	Female	21.0	1.52	56.0		yes	no	3.0	3.0	Some
2	Male	23.0	1.80	77.0		yes	no	2.0	3.0	Some
3	Male	27.0	1.80	87.0		no	no	3.0	3.0	Some
4	Male	22.0	1.78	89.8		no	no	2.0	1.0	Some

- Legendas de classe

Variable Role Type Description -----
 ----- Gender Feature Categorical Age Feature Continuous Height Feature Continuous Weight Feature Continuous
 family_history_with_overweight Feature Binary Has a family member suffered or suffers from overweight? FAVC Feature Binary
 Do you eat high caloric food frequently? FCVC Feature Integer Do you usually eat vegetables in your meals? NCP Feature
 Continuous How many main meals do you have daily? CAEC Feature Categorical Do you eat any food between meals? SMOKE
 Feature Binary Do you smoke? CH2O Feature Continuous How much water do you drink daily? SCC Feature Binary Do you
 monitor the calories you eat daily? FAF Feature Continuous How often do you have physical activity? TUE Feature Integer How
 much time do you use technological devices such as cell phone, videogames, television, computer and others? CALC Feature
 Categorical How often do you drink alcohol? MTRANS Feature Categorical Which transportation do you usually use?
 NObeyesdad Feature Categorical Obesity level

- Entendimento inicial

```
In [3]: rows, columns = df.shape
print(f" rows: {rows}, columns: {columns}")

rows: 2111, columns: 17
```

```
In [4]: df.dtypes
```

```
Out[4]: Gender          object
Age              float64
Height          float64
Weight          float64
family_history_with_overweight  object
FAVC            object
FCVC           float64
NCP            float64
CAEC           object
SMOKE          object
CH2O           float64
SCC            object
FAF            float64
TUE            float64
CALC           object
MTRANS         object
NObeyesdad     object
dtype: object
```

```
In [5]: print("Null Values by Feature")
print("-"*40)
```

```
df.isna().sum()
```

Null Values by Feature

```
-----
Out[5]:  Gender                                0
        Age                                    0
        Height                                0
        Weight                                0
        family_history_with_overweight        0
        FAVC                                  0
        FCVC                                  0
        NCP                                    0
        CAEC                                  0
        SMOKE                                 0
        CH20                                  0
        SCC                                    0
        FAF                                    0
        TUE                                    0
        CALC                                  0
        MTRANS                                0
        NObeyesdad                            0
        dtype: int64
```

```
In [6]: print(f"Duplicated values:{df.duplicated().sum()}")
        df = df.drop_duplicates().reset_index(drop=True)
        print(f"Duplicated values now:{df.duplicated().sum()}")
```

Duplicated values:24
Duplicated values now:0

- Identificação prévia de potenciais problemas

- Data leakage . Tudo ok. - Tipos errados (datas como string) . Tudo ok. - Variáveis com significado especial: . Na tarefa de clusterização transformamos as variáveis binárias ("yes"/no para 1,0). - Variáveis derivadas do target: . A tarefa está sendo tratada como clusterização.

Análise Exploratória

- A etapa de análise exploratória teve como objetivo validar a integridade dos dados, compreender as distribuições comportamentais e identificar relações latentes antes da aplicação dos algoritmos de clusterização. - Saneamento e Qualidade dos Dados: Inicialmente, verificou-se a consistência do dataset. Não foram encontrados valores ausentes (nulls), garantindo a robustez da amostra. Identificamos e removemos 24 registros duplicados, assegurando que cada observação representasse um indivíduo único. - Análise Univariada e Distribuições: A análise visual (histogramas e boxplots) revelou que variáveis como Age e NCP (número de refeições) possuem distribuições assimétricas. Identificamos outliers significativos nas variáveis de peso e IMC. Decisão estratégica: Manter os outliers, pois, no contexto de estudo da obesidade, os valores extremos (Obesidade Mórbida) não são erros de medição, mas sim o público-alvo de maior interesse para a segmentação de risco. - Análise Multivariada e Correlações: Para mapear as dependências entre as features, utilizamos duas abordagens estatísticas complementares: . Matriz de Correlação de Pearson (Numérica): Confirmou a multicolinearidade óbvia entre Weight, Height e IMC. Como ação corretiva para a modelagem, optou-se por utilizar o IMC como variável representativa, descartando as variáveis originais de peso e altura para evitar redundância no cálculo de distâncias; . V de Cramér Corrigido (Categórica): Aplicamos esta métrica para quantificar a associação entre variáveis qualitativas. O teste revelou uma associação forte ($V > 0.5$) entre Family_History (Histórico Familiar) e o nível de obesidade, validando a hipótese de que a predisposição genética é um fator discriminante crucial.

- Dataset EDA

```
In [7]: # Dataset para análise exploratória
df_eda = df.copy()
```

- Transformações no dataset

```
In [8]: # Com altura e peso temos o IMC
df_eda["BMI"] = df_eda["Weight"]/df_eda["Height"]**2
```

- Descrição do dataset

```
In [9]: df_eda.describe().round(4)
```

```
Out[9]:
```

	Age	Height	Weight	FCVC	NCP	CH2O	FAF	T
count	2087.0000	2087.0000	2087.0000	2087.0000	2087.0000	2087.0000	2087.0000	2087.0000
mean	24.3531	1.7027	86.8587	2.4215	2.7012	2.0047	1.0128	0.6000
std	6.3688	0.0932	26.1908	0.5347	0.7646	0.6083	0.8535	0.6000
min	14.0000	1.4500	39.0000	1.0000	1.0000	1.0000	0.0000	0.0000
25%	19.9159	1.6302	66.0000	2.0000	2.6975	1.5909	0.1245	0.0000
50%	22.8476	1.7016	83.1011	2.3963	3.0000	2.0000	1.0000	0.6000
75%	26.0000	1.7695	108.0159	3.0000	3.0000	2.4662	1.6781	1.0000
max	61.0000	1.9800	173.0000	3.0000	4.0000	3.0000	3.0000	2.0000

```
In [10]: cols_bin = ["Gender", "family_history_with_overweight", "FAVC", "SMOKE", "SCC"]

cols_cat = ["CAEC", "CALC", "MTRANS", "NObeyesdad"]

cols_num = ["Age", "BMI", "FCVC", "NCP", "CH2O", "FAF", "TUE"]
```

- Heatmap gênero x obesidade (IMC)

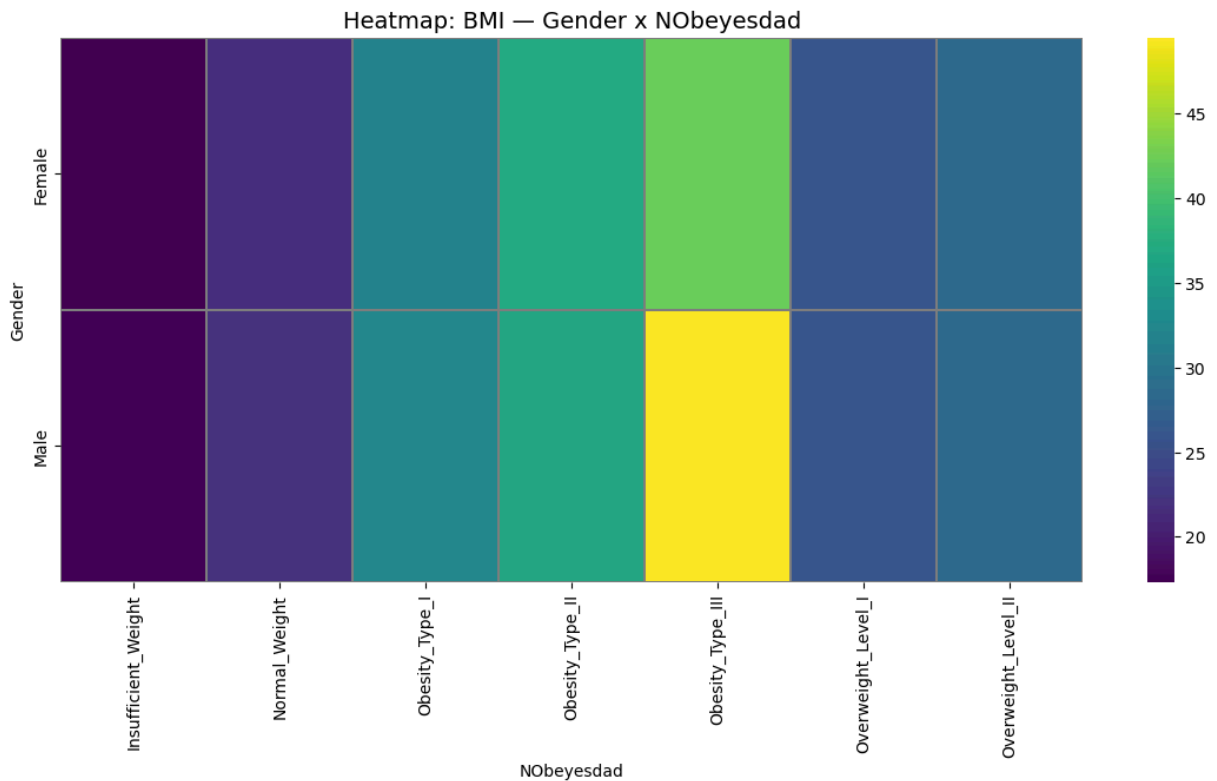
- Homens com maior IMC, em média, estão mais associados a obesidade tipo 3.

```
In [11]: pivot = df_eda.pivot_table(
    values="BMI",
    index="Gender",
    columns="NObeyesdad",
    aggfunc="mean"
)

plt.figure(figsize=(14,6))
sns.heatmap(
    pivot,
    cmap="viridis",
    linewidths=0.1,
    linecolor="gray"
```

```
)

plt.title("Heatmap: BMI — Gender x NObeyesdad", fontsize=14)
plt.xlabel("NObeyesdad")
plt.ylabel("Gender")
plt.show()
```



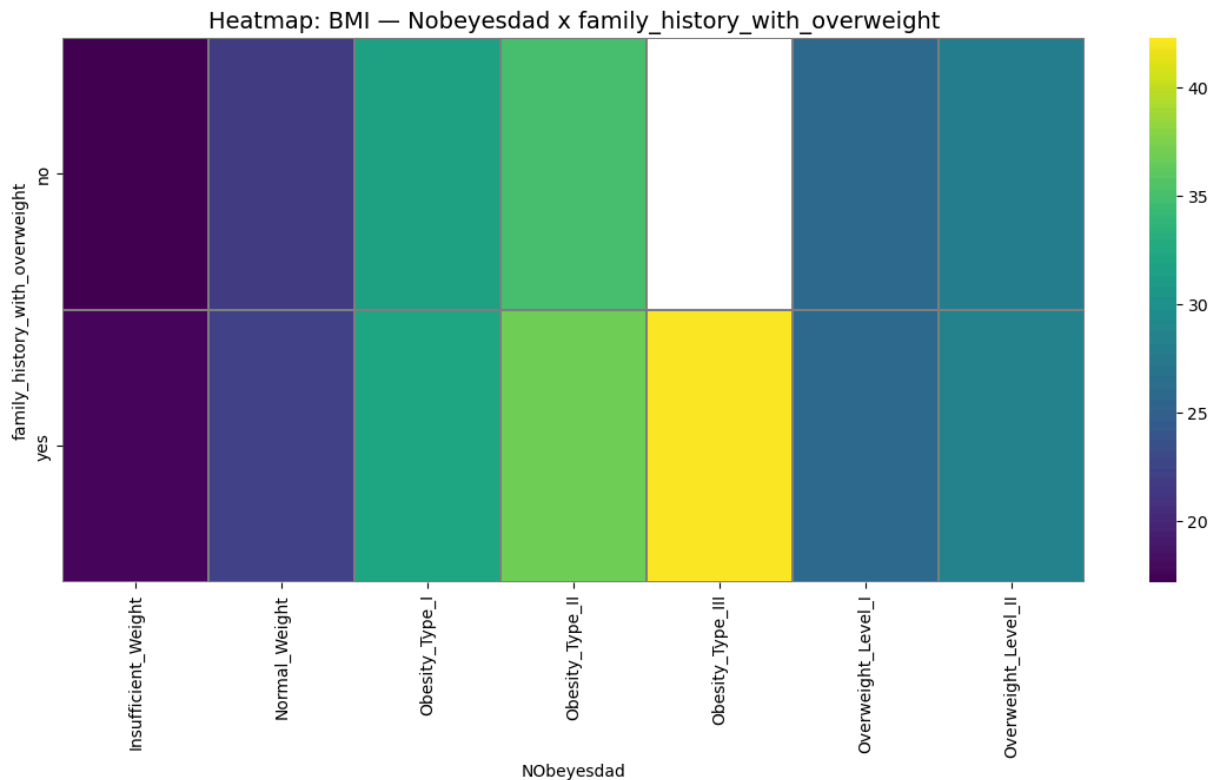
- Heatmap histórico familiar de sobrepeso x obesidade (IMC)

- Indivíduos com maior IMC e histórico familiar de sobrepeso, em média, estão mais associados a casos de obesidade tipo 3

```
In [12]: pivot = df_eda.pivot_table(
    values="BMI",
    index="family_history_with_overweight",
    columns="NObeyesdad",
    aggfunc="mean"
)

plt.figure(figsize=(14,6))
sns.heatmap(
    pivot,
    cmap="viridis",
    linewidths=0.1,
    linecolor="gray"
)

plt.title("Heatmap: BMI — Nobeyesdad x family_history_with_overweight", fontsize=14)
plt.xlabel("NObeyesdad")
plt.ylabel("family_history_with_overweight")
plt.show()
```



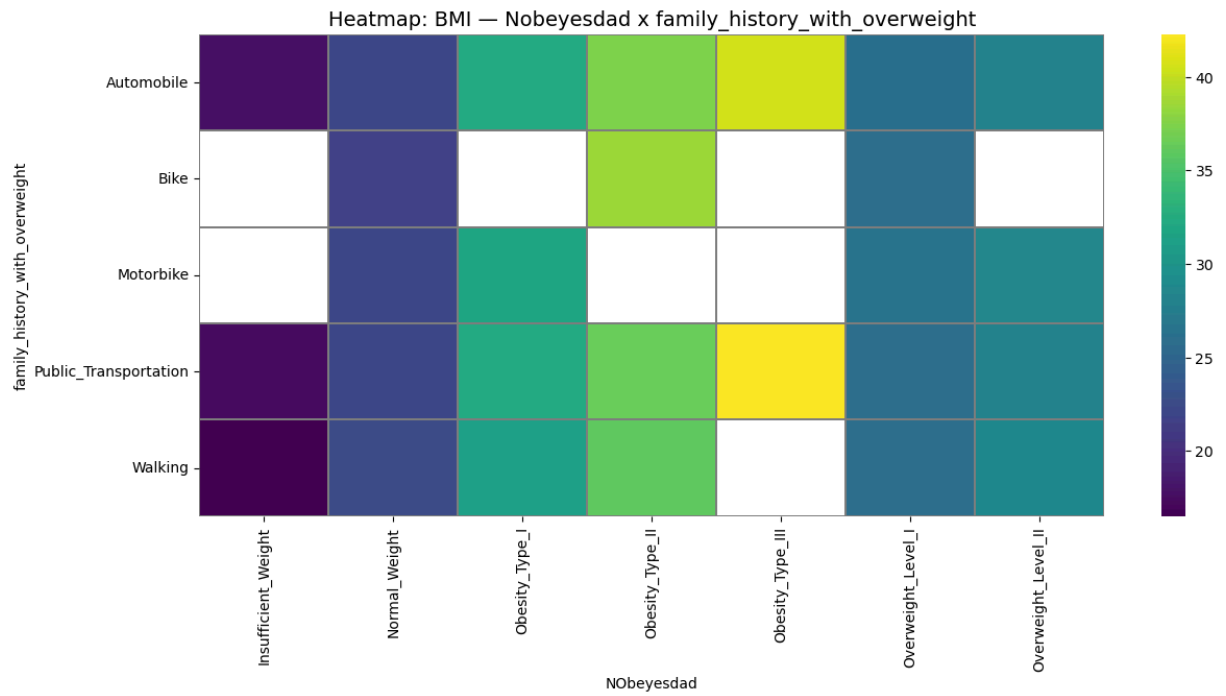
- Heatmap tipo de transporte x obesidade (IMC)

- Indivíduos com maior IMC e que utilizam o transporte público para se locomover (usualmente), em média, estão mais associados a obesidade tipo 3.

```
In [13]: pivot = df_eda.pivot_table(
    values="BMI",
    index="MTRANS",
    columns="Nobeyesdad",
    aggfunc="mean"
)

plt.figure(figsize=(14,6))
sns.heatmap(
    pivot,
    cmap="viridis",
    linewidths=0.1,
    linecolor="gray"
)

plt.title("Heatmap: BMI — Nobeyesdad x family_history_with_overweight", fontsize=14)
plt.xlabel("Nobeyesdad")
plt.ylabel("family_history_with_overweight")
plt.show()
```



- Distribuição das variáveis numéricas

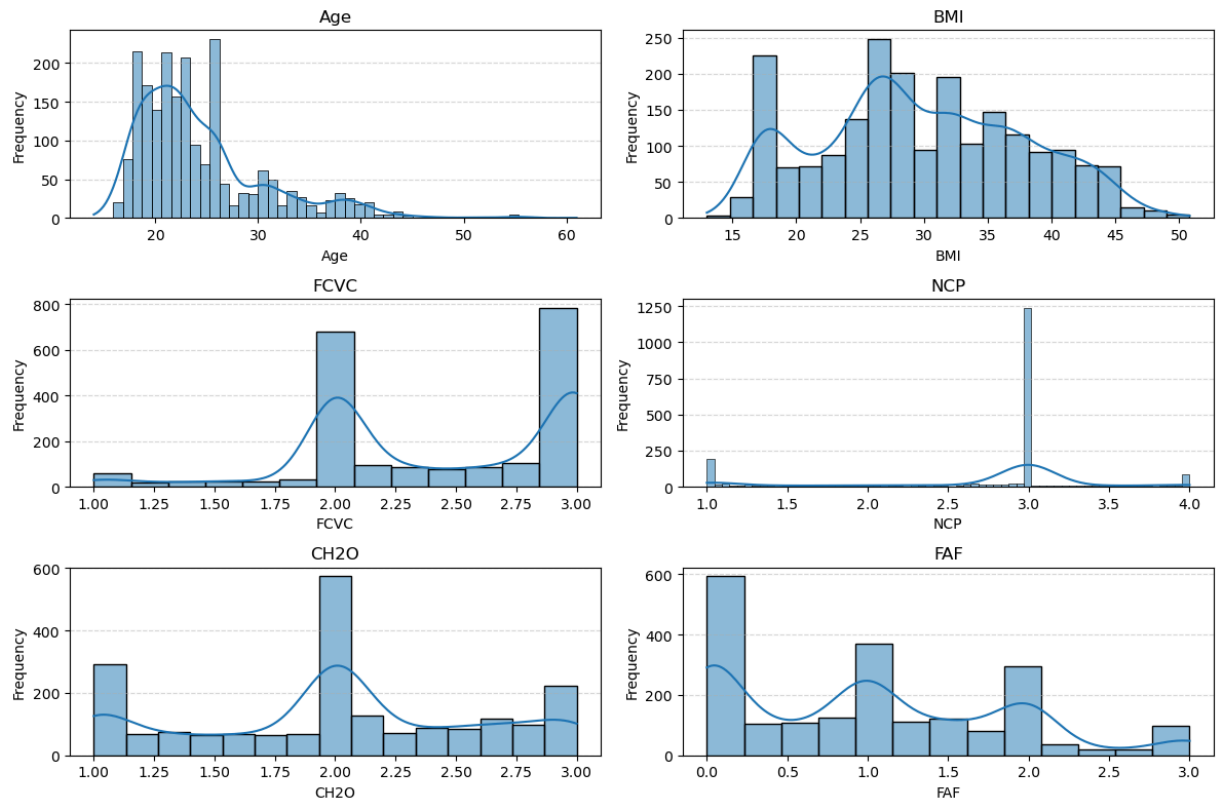
- Essas variáveis tem caráter discreto. "BMI" e "FAF" são bimodais. - "Age" e "NCP" são as variáveis que possuem uma quantidade considerável de outliers.

```
In [14]: fig, axes = plt.subplots(3, 2, figsize=(12, 8))
axes = axes.flatten()

for ax, col in zip(axes, cols_num):
    sns.histplot(
        data=df_eda,
        x=col,
        kde=True,
        edgecolor="black",
        ax=ax
    )
    ax.set_title(col)
    ax.set_xlabel(col)
    ax.set_ylabel("Frequency")
    ax.grid(axis="y", linestyle="--", alpha=0.5)

# Caso tenha mais axes do que colunas numéricas
for ax in axes[len(cols_num):]:
    fig.delaxes(ax)

plt.tight_layout()
plt.show()
```

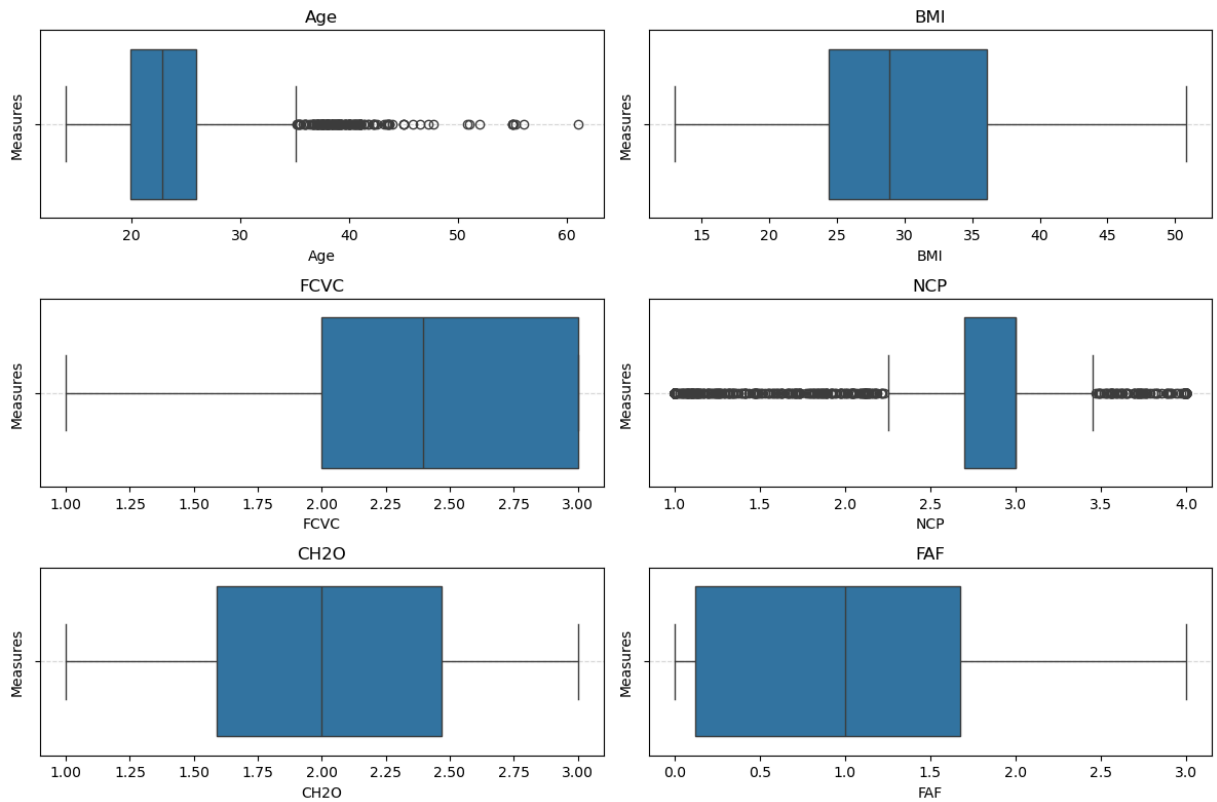



```
In [15]: fig, axes = plt.subplots(3, 2, figsize=(12, 8))
axes = axes.flatten()

for ax, col in zip(axes, cols_num):
    sns.boxplot(
        data=df_eda,
        x=col,
        ax=ax
    )
    ax.set_title(col)
    ax.set_xlabel(col)
    ax.set_ylabel("Measures")
    ax.grid(axis="y", linestyle="--", alpha=0.5)

# Caso tenha mais axes do que colunas numéricas
for ax in axes[len(cols_num):]:
    fig.delaxes(ax)

plt.tight_layout()
plt.show()
```



- Distribuição das variáveis categóricas

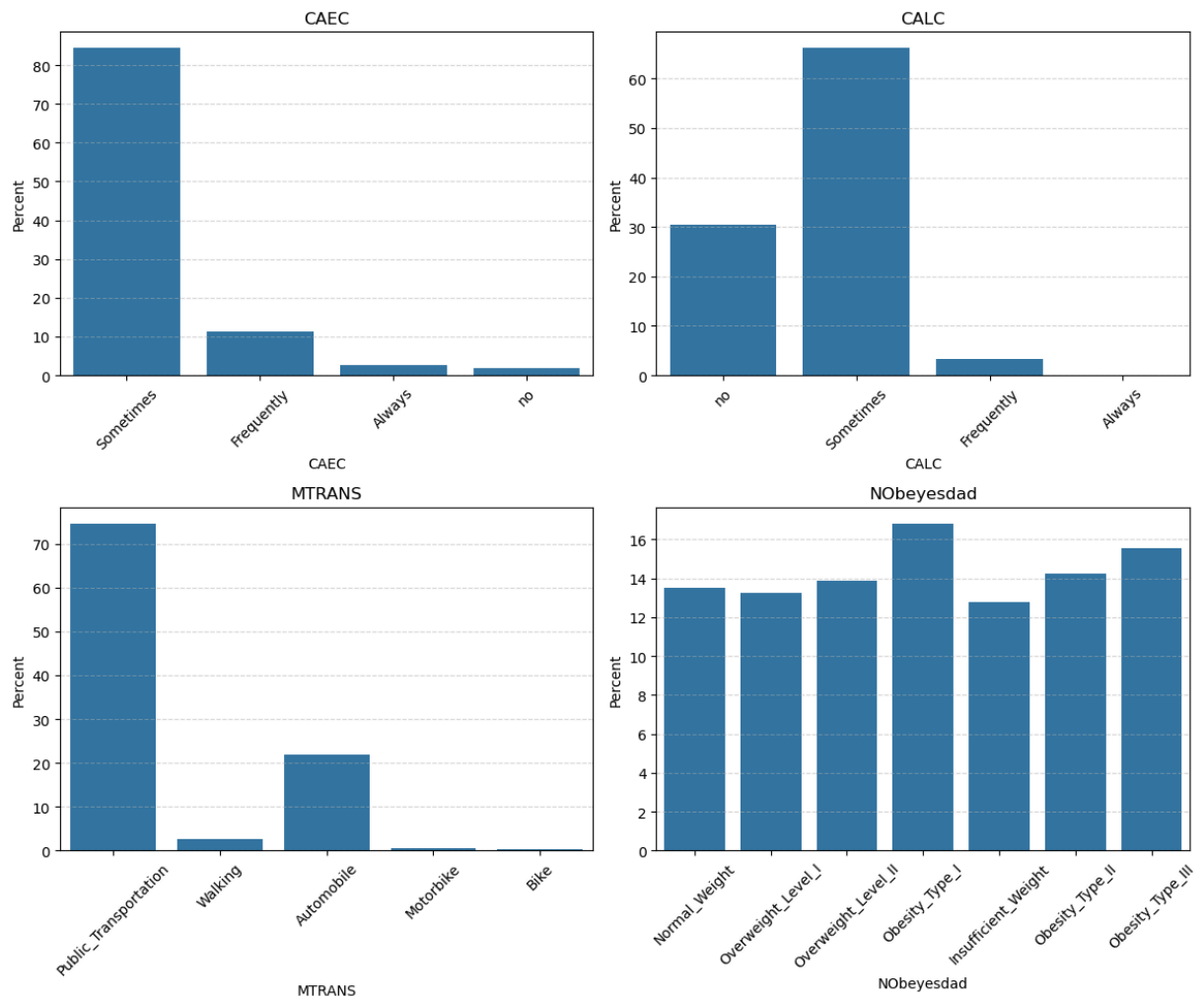
- Apenas a variável "NObedesdad" apresenta equilíbrio em suas classes.

```
In [16]: fig, axes = plt.subplots(2, 2, figsize=(12, 10))
axes = axes.flatten()

for ax, col in zip(axes, cols_cat):
    sns.countplot(
        data=df_eda,
        x=col,
        stat="percent",
        ax=ax
    )
    ax.set_title(col)
    ax.set_xlabel(col)
    ax.set_ylabel("Percent")
    ax.grid(axis="y", linestyle="--", alpha=0.5)
    ax.tick_params("x", rotation=45)

# Caso tenha mais axes do que colunas numéricas
for ax in axes[len(cols_cat):]:
    fig.delaxes(ax)

plt.tight_layout()
plt.show()
```



- Distribuição das variáveis binárias

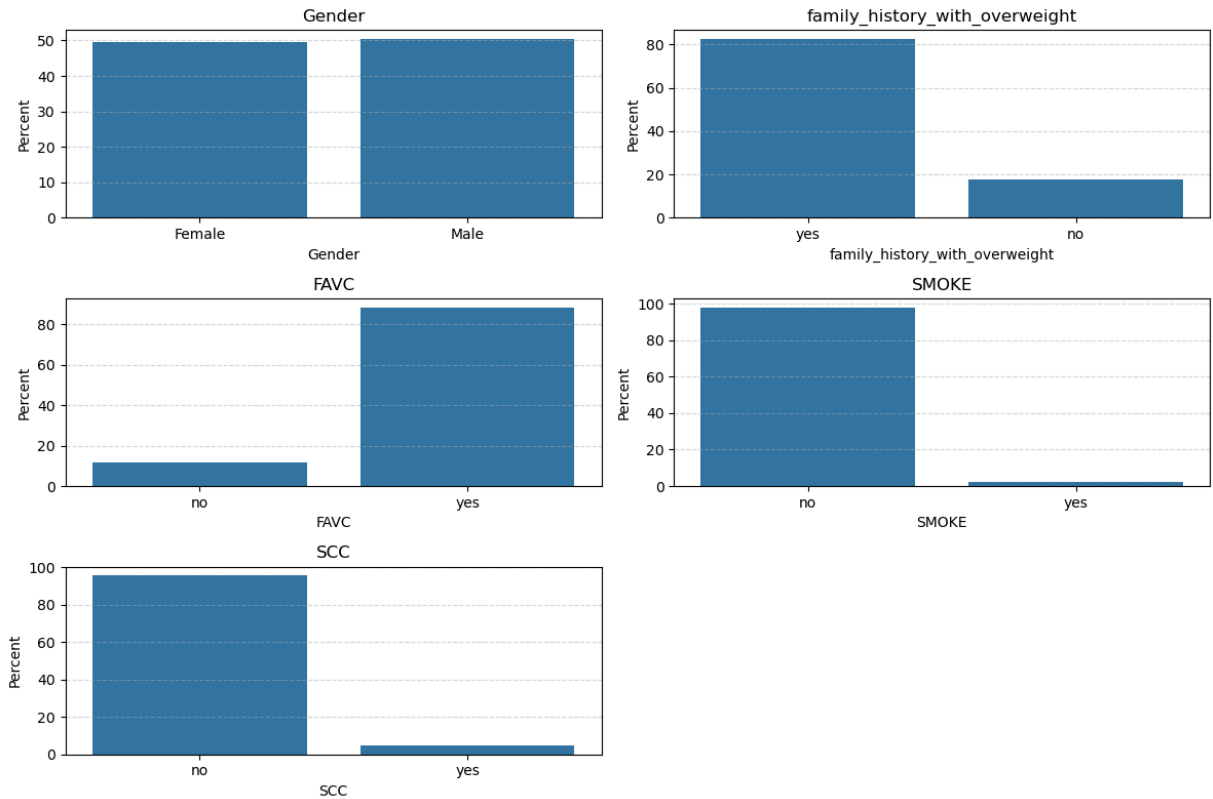
- Somente "Gender" é equilibrada em suas classes.

```
In [17]: fig, axes = plt.subplots(3, 2, figsize=(12, 8))
axes = axes.flatten()

for ax, col in zip(axes, cols_bin):
    sns.countplot(
        data=df_eda,
        x=col,
        stat="percent",
        ax=ax
    )
    ax.set_title(col)
    ax.set_xlabel(col)
    ax.set_ylabel("Percent")
    ax.grid(axis="y", linestyle="--", alpha=0.5)

# Caso tenha mais axes do que colunas numéricas
for ax in axes[len(cols_bin):]:
    fig.delaxes(ax)
```

```
plt.tight_layout()
plt.show()
```

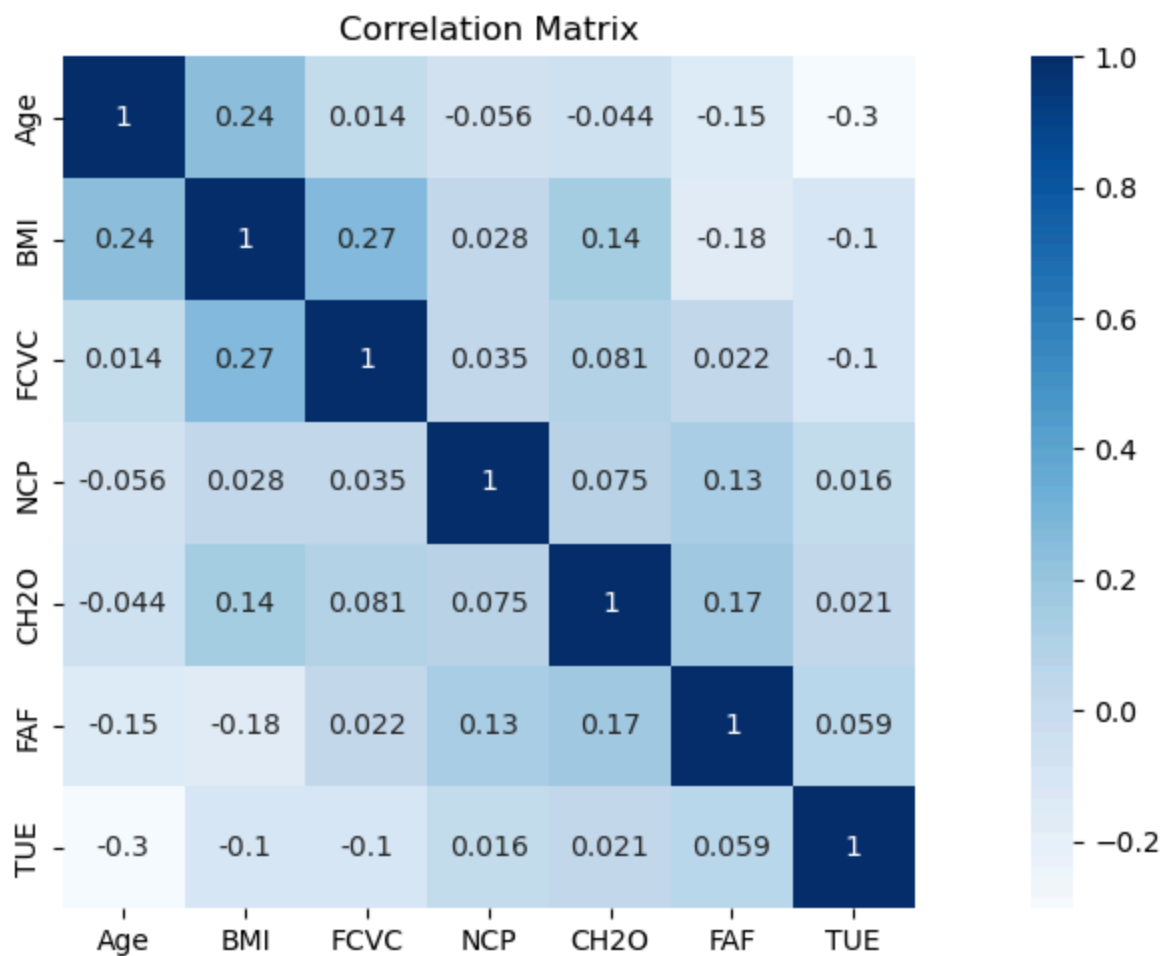


- Corelação

- A relação entre as variáveis é fraca.

```
In [18]: corr = df_eda[cols_num].corr().round(4)
plt.figure(figsize=(12,5))
plt.title("Correlation Matrix")
sns.heatmap(corr, vmax=1, square="True", annot=True, cmap="Blues")

plt.tight_layout()
plt.show()
```



- Associação

Valor do V de Cramér Interpretação da Associação Comentário de Negócio -----
 ----- 0.00 a 0.10 Desprezível / Nula A variável não ajuda a distinguir as classes. Ruído. 0.10 a 0.30 Fraca Existe um sinal leve, mas sozinha a variável não decide nada. Útil em conjunto com outras. 0.30 a 0.50 Moderada Associação Importante. Esta variável já tem poder preditivo relevante. > 0.50 Forte Determinante. A variável quase "define" o resultado. (Raro em dados sociais/humanos).- A associação entre "Gender" e "NObeyesdad" é forte (0.56) - A associação entre "family_history_with_overweight" e "NObeyesdad" também é forte (0.54).

```
In [19]: def cramers_v_corrected(confusion_matrix):
chi2 = stats.chi2_contingency(confusion_matrix)[0]
n = confusion_matrix.sum().sum()
phi2 = chi2 / n
r, k = confusion_matrix.shape

phi2corr = max(0, phi2 - ((k - 1)*(r - 1))/(n - 1))
rcorr = r - (r - 1)**2 / (n - 1)
kcorr = k - (k - 1)**2 / (n - 1)

return np.sqrt(phi2corr / min((kcorr - 1), (rcorr - 1)))

cols_v = cols_bin + cols_cat

mat = pd.DataFrame(index=cols_v, columns=cols_v)

for c1 in cols_v:
```

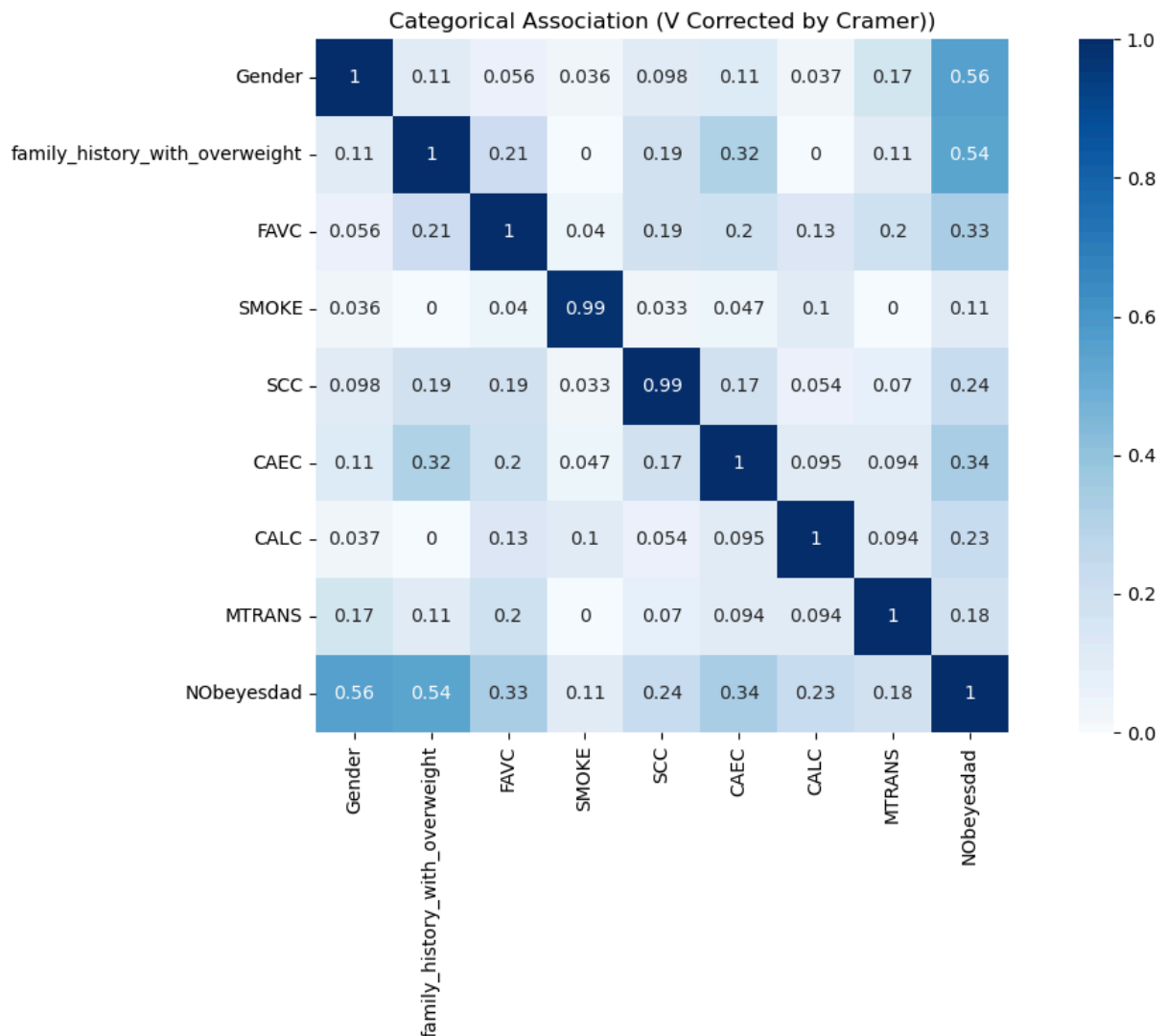
```

for c2 in cols_v:
    tab = pd.crosstab(df_eda[c1], df_eda[c2])
    mat.loc[c1, c2] = cramers_v_corrected(tab.values)

plt.figure(figsize=(12,8))
sns.heatmap(mat.astype(float), annot=True, square=True, cmap='Blues', vmin=0, vmax=
plt.title("Categorical Association (V Corrected by Cramer)")

plt.tight_layout()
plt.show()

```



Pré-processamento

A preparação dos dados foi desenhada para garantir a equidade geométrica entre as variáveis, visto que algoritmos de clusterização são sensíveis à escala e à multicolinearidade. O pipeline de transformação consistiu em quatro etapas estratégicas: . Definição do Escopo (Unsupervised Learning): Como o objetivo é descobrir agrupamentos naturais e não prever uma classe pré-existente, a variável alvo original (NObeyesdad) foi removida do conjunto de treino. Ela foi reservada apenas para uso posterior na validação externa dos clusters; . Engenharia de Atributos (Feature Engineering): Para capturar a relação não-linear entre peso e altura em uma única métrica representativa, calculou-se o Índice de Massa Corporal (IMC). Consequentemente, as variáveis originais Weight e Height foram descartadas para eliminar a multicolinearidade estrutural e reduzir a redundância no cálculo das distâncias euclidianas; . Codificação de Variáveis Categóricas: Aplicamos tratamentos distintos baseados na natureza das variáveis qualitativas: * Variáveis Binárias: Mapeamento direto (0/1) para atributos como Gender, Family_history e SMOKE3333; *

Variáveis Nominais: Aplicação de OneHotEncoder em variáveis como Transporte (MTRANS) e Consumo de Álcool (CALC). Optou-se por não descartar a primeira coluna (drop_first=False) para preservar a dimensionalidade completa do espaço de características, uma prática recomendada em clusterização para não enviesar a geometria espacial; * Padronização de Escala (Scaling): Dada a presença de variáveis com magnitudes díspares (ex: Age em dezenas vs. NCP em unidades), aplicou-se o StandardScaler em todas as variáveis numéricas. Esta etapa foi crítica para colocar todas as features na mesma escala de desvio padrão, impedindo que variáveis de maior magnitude dominassem a função de inércia dos modelos. - Arquitetura do Pipeline: Todas as transformações foram encapsuladas em um objeto ColumnTransformer dentro de um Pipeline do Scikit-Learn, garantindo reprodutibilidade e evitando vazamento de dados (data leakage) durante os experimentos

- Dataset de pré-processamento

```
In [20]: df_prep = df_eda.copy()
```

- Remover o rótulo real

```
In [21]: df_prep = df_prep.drop(columns=["NObeyesdad"]).copy().reset_index(drop=True)
```

- Padronizar binárias (yes/no → 0/1)

```
In [22]: bin_map = {
    "Gender": {"Female": 1, "Male": 0},
    "family_history_with_overweight": {"yes": 1, "no": 0},
    "FAVC": {"yes": 1, "no": 0},
    "SMOKE": {"yes": 1, "no": 0},
    "SCC": {"yes": 1, "no": 0},
}
```

```
In [23]: for col, mapping in bin_map.items():
    df_prep[col] = df_prep[col].map(mapping)
```

- Features irrelevantes

```
In [24]: # Removendo as variáveis colineares
df_prep = df_prep.drop(columns=["Height", "Weight"]).reset_index(drop=True)
```

- Classificação das variáveis

```
In [25]: # Binárias já convertidas para 0/1
cols_bin = ["Gender", "family_history_with_overweight", "FAVC", "SMOKE", "SCC"]

# Categóricas nominais para one-hot
# Clusterização depende de distâncias no espaço vetorial
cols_cat = ["CAEC", "CALC", "MTRANS"]

# Numéricas contínuas (incluindo IMC)
cols_num = ["Age", "FCVC", "NCP", "CH20", "FAF", "TUE", "BMI"]
```

- Ajustar os tipos numéricos

```
In [26]: df_prep[cols_num] = df_prep[cols_num].astype(float)
```

Modelos Base com Pipelines

- Para identificar a estrutura latente mais robusta, estabeleceu-se um framework experimental rigoroso. Utilizou-se a arquitetura de Pipelines do Scikit-Learn para encapsular o pré-processamento e a modelagem em um fluxo único, garantindo a reprodutibilidade e prevenindo vazamento de dados (data leakage) em todas as iterações. - Diversidade de Algoritmos: Testamos quatro famílias de algoritmos para cobrir diferentes premissas geométricas de clusterização: . K-Means: Baseado em centróides e distâncias euclidianas (assume clusters esféricos); . Gaussian Mixture Models (GMM): Baseado em probabilidade e covariância (assume clusters elípticos e flexíveis); . Agglomerative Clustering: Baseado em hierarquia e conectividade; . BIRCH: Processar grandes volumes de dados e mitigar outliers; . K-Means++: Idêntico ao K-Means, mas utiliza um algoritmo probabilístico para "espalhar" os centroides iniciais antes de começar a iterar. - Otimização de Hiperparâmetros: Foi implementada uma função de avaliação universal ("Avaliador de Pipeline") para iterar automaticamente sobre o número de clusters (k) variando de 2 a 9. Essa abordagem permitiu testar a estabilidade da segmentação em diferentes níveis de granularidade. - Critérios de Seleção (Métricas Internas): Como não há "rótulos verdadeiros" no aprendizado não supervisionado, a seleção do modelo campeão baseou-se no consenso de três métricas de validação interna: . Silhouette Score: Mede o quão similar um objeto é ao seu próprio cluster em comparação aos outros (foco em coesão e separação); . Davies-Bouldin Index: Avalia a similaridade média entre cada cluster e seu cluster mais similar (quanto menor, melhor); . Calinski-Harabasz Score: Razão entre a dispersão intra-cluster e inter-cluster (foco em definição densa). - O K-Means com k=5 emergiu como o modelo vencedor, apresentando o melhor equilíbrio entre pontuação de Silhouette e interpretabilidade de negócio.

- Definir X e o pipeline de pré-processamento

```
In [27]: # Matriz de atributos para clusterização (sem NObeyesdad)
X = df_prep.copy()

# Pré-processamento em pipeline
# Em classificação/regressão, usamos drop='first' para evitar multicolinearidade.
# Isso não é necessário em uma tarefa de clusterização.
preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), cols_num),
        ("bin", "passthrough", cols_bin), # já estão em 0/1
        ("cat", OneHotEncoder(drop=None, handle_unknown="ignore"), cols_cat),
    ],
    remainder="drop"
)
```

- Criar um avaliador universal

```
In [28]: def evaluate_cluster_pipeline(name, pipeline, X):
    """
    Ajusta o pipeline, transforma os dados e calcula métricas de clusterização
    para modelos baseados em k (KMeans, Birch, Agglo, GMM).
    """
    # 1. Transformação e Predição
    # O fit_transform garante que a escala e o encoding sejam aplicados corretament
    X_trans = pipeline[:-1].fit_transform(X)

    # O último passo do pipeline é o modelo de cluster
    model = pipeline.steps[-1][1]
```



```

# Alguns modelos (GMM) usam .predict(), outros .labels_ ou .fit_predict()
if hasattr(model, "fit_predict"):
    labels = model.fit_predict(X_trans)
else:
    # Caso específico para Gaussian Mixture que não possui fit_predict em algum
    labels = model.fit(X_trans).predict(X_trans)

# 2. Contagem de clusters encontrados
n_clusters = len(np.unique(labels))

# 3. Cálculo das métricas (exige no mínimo 2 clusters e no máximo N-1)
if 1 < n_clusters < X_trans.shape[0]:
    sil = silhouette_score(X_trans, labels)
    db = davies_bouldin_score(X_trans, labels)
    ch = calinski_harabasz_score(X_trans, labels)
else:
    sil, db, ch = np.nan, np.nan, np.nan

return {
    "Model": name,
    "Silhouette": sil,
    "Davies-Bouldin": db,
    "Calinski-Harabasz": ch,
    "n_clusters": n_clusters
}

```

- Pipeline para cada método

```

In [29]: results = []

# KMeans
for k in range(2, 10):
    kmeans_pipe = Pipeline(steps=[
        ("preprocess", preprocessor),
        ("cluster", KMeans(n_clusters=k, random_state=42))
    ])
    res = evaluate_cluster_pipeline(f"KMeans_k={k}", kmeans_pipe, X)
    results.append(res)

# Gaussian Mixture (GMM)
for k in range(2, 10):
    gmm_pipe = Pipeline(steps=[
        ("preprocess", preprocessor),
        ("cluster", GaussianMixture(n_components=k, random_state=42))
    ])
    res = evaluate_cluster_pipeline(f"GMM_k={k}", gmm_pipe, X)
    results.append(res)

# Agglomerative Clustering
for k in range(2, 10):
    agglo_pipe = Pipeline(steps=[
        ("preprocess", preprocessor),
        ("cluster", AgglomerativeClustering(n_clusters=k))
    ])
    res = evaluate_cluster_pipeline(f"Agglo_k={k}", agglo_pipe, X)

```

```

        results.append(res)

# BIRCH
for k in range(2, 10):
    birch_pipe = Pipeline(steps=[
        ("preprocess", preprocessor),
        ("cluster", Birch(n_clusters=k))
    ])
    res = evaluate_cluster_pipeline(f"Birch_k={k}", birch_pipe, X)
    results.append(res)

# K-means++ (similar ao KMedoids)
for k in range(2, 10):
    kmeans_pp_pipe = Pipeline(steps=[
        ("preprocess", preprocessor),
        ("cluster", KMeans(n_clusters=k, init='k-means++', random_state=42))
    ])
    res = evaluate_cluster_pipeline(f"KMeans++_k={k}", kmeans_pp_pipe, X)
    results.append(res)

```

```

File "C:\Users\rodri\anaconda3\Lib\site-packages\joblib\externals\loky\backend\con
text.py", line 257, in _count_physical_cores
    cpu_info = subprocess.run(
        "wmic CPU Get NumberOfCores /Format:csv".split(),
        capture_output=True,
        text=True,
    )
File "C:\Users\rodri\anaconda3\Lib\subprocess.py", line 554, in run
    with Popen(*popenargs, **kwargs) as process:
        ~~~~~^~~~~~
File "C:\Users\rodri\anaconda3\Lib\subprocess.py", line 1039, in __init__
    self._execute_child(args, executable, preexec_fn, close_fds,
    ~~~~~^~~~~~
        pass_fds, cwd, env,
        ~~~~~^~~~~~
    ...<5 lines>...
        gid, gids, uid, umask,
        ~~~~~^~~~~~
        start_new_session, process_group)
        ~~~~~^~~~~~
File "C:\Users\rodri\anaconda3\Lib\subprocess.py", line 1554, in _execute_child
    hp, ht, pid, tid = _winapi.CreateProcess(executable, args,
    ~~~~~^~~~~~
        # no special security
        ~~~~~^~~~~~
    ...<4 lines>...
        cwd,
        ~~~~~^~~~~~
        startupinfo)
        ~~~~~^~~~~~

```

- Compilar resultados

```

In [30]: df_results = pd.DataFrame(results)
df_results.sort_values("Silhouette", ascending=False, inplace=True)

```

df_results

Out[30]:

	Model	Silhouette	Davies-Bouldin	Calinski-Harabasz	n_clusters
35	KMeans++_k=5	0.167864	1.931196	298.894770	5
3	KMeans_k=5	0.167864	1.931196	298.894770	5
2	KMeans_k=4	0.162013	1.866014	312.186063	4
34	KMeans++_k=4	0.162013	1.866014	312.186063	4
39	KMeans++_k=9	0.153449	1.803450	227.733267	9
7	KMeans_k=9	0.153449	1.803450	227.733267	9
6	KMeans_k=8	0.151200	1.703975	242.659479	8
38	KMeans++_k=8	0.151200	1.703975	242.659479	8
27	Birch_k=5	0.146263	2.113702	265.069262	5
29	Birch_k=7	0.144047	2.169490	212.316722	7
28	Birch_k=6	0.143604	2.102236	237.243403	6
25	Birch_k=3	0.141563	2.093150	242.637648	3
30	Birch_k=8	0.140186	2.268359	194.897333	8
18	Agglo_k=4	0.139855	1.924303	265.531076	4
1	KMeans_k=3	0.136778	1.977241	309.483816	3
33	KMeans++_k=3	0.136778	1.977241	309.483816	3
19	Agglo_k=5	0.131842	2.254878	246.971933	5
24	Birch_k=2	0.129933	2.213268	246.183860	2
5	KMeans_k=7	0.129835	1.835256	243.848546	7
37	KMeans++_k=7	0.129835	1.835256	243.848546	7
22	Agglo_k=8	0.125968	2.032117	200.625631	8
4	KMeans_k=6	0.124780	1.908529	257.623457	6
36	KMeans++_k=6	0.124780	1.908529	257.623457	6
31	Birch_k=9	0.123520	2.298274	182.160189	9
21	Agglo_k=7	0.121695	2.120454	214.274018	7
17	Agglo_k=3	0.121208	1.886857	270.696176	3
26	Birch_k=4	0.119000	2.305245	219.979859	4
23	Agglo_k=9	0.117556	1.967175	187.781767	9
16	Agglo_k=2	0.116570	2.528733	266.600028	2
20	Agglo_k=6	0.112393	2.149159	226.184738	6

	Model	Silhouette	Davies-Bouldin	Calinski-Harabasz	n_clusters
32	KMeans++_k=2	0.108331	2.826440	246.999374	2
0	KMeans_k=2	0.108331	2.826440	246.999374	2
8	GMM_k=2	0.101187	5.737463	17.170959	2
9	GMM_k=3	0.095991	5.885223	86.761847	3
10	GMM_k=4	0.081579	4.681509	73.191987	4
13	GMM_k=7	0.070632	4.576747	84.470958	7
11	GMM_k=5	0.063055	3.908888	59.998008	5
14	GMM_k=8	0.054511	3.406918	76.004005	8
15	GMM_k=9	0.050692	3.385266	67.369196	9
12	GMM_k=6	0.019285	3.704778	62.278054	6

Modelo Escolhido: K-Means com k = 5

- O cluster 0 contém os indivíduos definidos como obesos grau 1, que fazem, em média, 3 refeições por dia e têm 35 anos. - O cluster 1 contém os indivíduos definidos como normais/saudáveis, que fazem, em média, 3 refeições por dia e tem 21 anos. - O cluster 2 contém os indivíduos que têm sobrepeso, que fazem, em média, 3 refeições por dia e tem 21 anos. - O cluster 3 contém os indivíduos que têm sobrepeso, que fazem, em média, 1 refeição por dia e tem 22 anos. - O cluster 4 contém os indivíduos que têm obesidade grau 3, que fazem, em média, 3 refeições por dia e tem 24 anos.- Interpretação do heatmap dos centróides: . Cluster 0: Os mais velhos; . Cluster 1: Os mais "saudáveis"; . Cluster 2: Os que comem menos vegetais; . Cluster 3: Os que têm menos refeições no dia; . Cluster 4: Os menos "saudáveis". - Interpretação do Cohen's d: . As variáveis NCP (quantas refeições você faz no dia), idade, IMC, FCVC (quantas vezes você come vegetais na refeição) e FAF (quantas vezes você faz atividade física) discriminam os clusters; . Já TUE (frequência de uso de aparelhos tecnológicos) e CH20 (quantidade de água ingerida no dia) tem força moderada na separação dos clusters.

- Reajustar o modelo vencedor no dataset preprocessado

```
In [31]: X_prep = preprocessor.fit_transform(df_prep)

best_k = 5
kmeans = KMeans(n_clusters=best_k, random_state=42)
clusters = kmeans.fit_predict(X_prep)

df_clusters = df_prep.copy()
df_clusters["cluster"] = clusters
```

- Perfil dos clusters

```
In [32]: # Médias por cluster (para variáveis contínuas)
df_clusters.groupby("cluster")[cols_num].mean()
```

Out[32]:

	Age	FCVC	NCP	CH2O	FAF	TUE	BMI
cluster							
0	35.148737	2.360756	2.626022	1.847437	0.940670	0.298208	31.305725
1	20.542272	2.583149	3.190044	2.184740	1.934706	0.791803	22.591433
2	21.452410	1.949413	2.987717	1.852083	0.680756	0.848508	27.133028
3	21.733139	2.365535	1.178030	1.896602	0.762115	0.746095	26.883680
4	23.925790	2.918985	2.970378	2.222073	0.764531	0.577112	40.264561

```
In [33]: # Distribuições de categorias por cluster
df_clusters.groupby("cluster")[cols_cat+cols_bin].agg(lambda x: x.value_counts(norm
```

Out[33]:

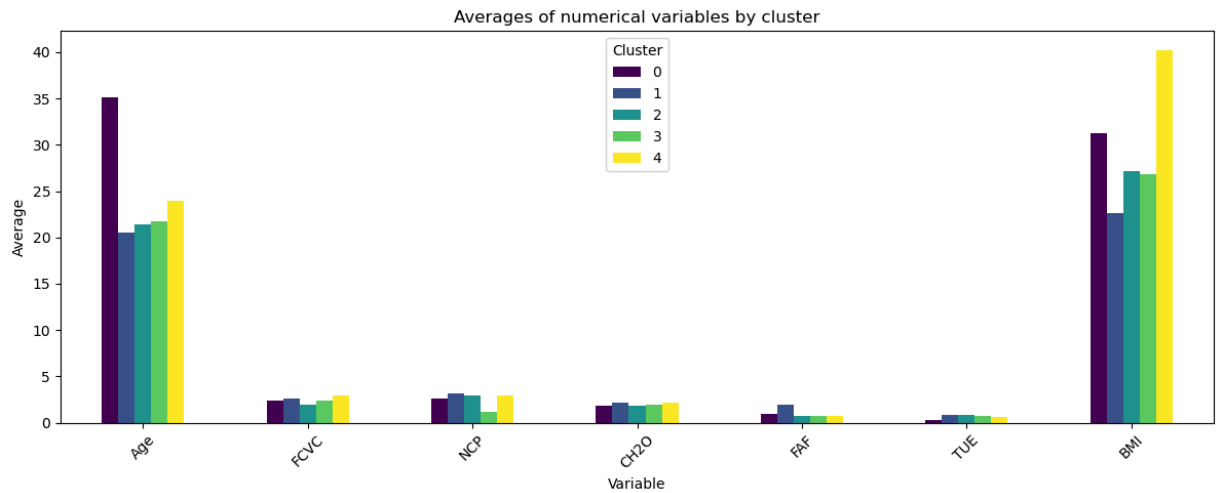
	CAEC	CALC	MTRANS	Gender	family_history_with_overweight	FAVC
cluster						
0	0.939948	0.618799	0.819843	0.605744		0.929504 0.916449 (
1	0.674528	0.474057	0.740566	0.570755		0.693396 0.764151 (
2	0.820225	0.677903	0.893258	0.651685		0.779026 0.883895 (
3	0.771044	0.498316	0.919192	0.612795		0.710438 0.855219 (
4	0.997773	0.982183	0.964365	0.743875		0.991091 0.986637 (

- Gráficos

```
In [34]: # Barplot das médias das variáveis por cluster
cluster_means = df_clusters.groupby("cluster")[cols_num].mean()

plt.figure(figsize=(12, 5))
cluster_means.T.plot(kind='bar', figsize=(12,5), colormap="viridis")
plt.title("Averages of numerical variables by cluster")
plt.ylabel("Average")
plt.xlabel("Variable")
plt.xticks(rotation=45)
plt.legend(title="Cluster")
plt.tight_layout()
plt.show();
```

<Figure size 1200x500 with 0 Axes>



```
In [35]: # Heatmap dos centróides
# Recuperar nomes numéricos (sem alteração)
num_features = cols_num.copy()

# Recuperar nomes das bins (OneHotEncoder)
bin_encoder = preprocessor.named_transformers_["bin"]
bin_features = bin_encoder.get_feature_names_out(cols_bin)

# Recuperar nomes das categóricas (OneHotEncoder)
cat_encoder = preprocessor.named_transformers_["cat"]
cat_features = cat_encoder.get_feature_names_out(cols_cat)

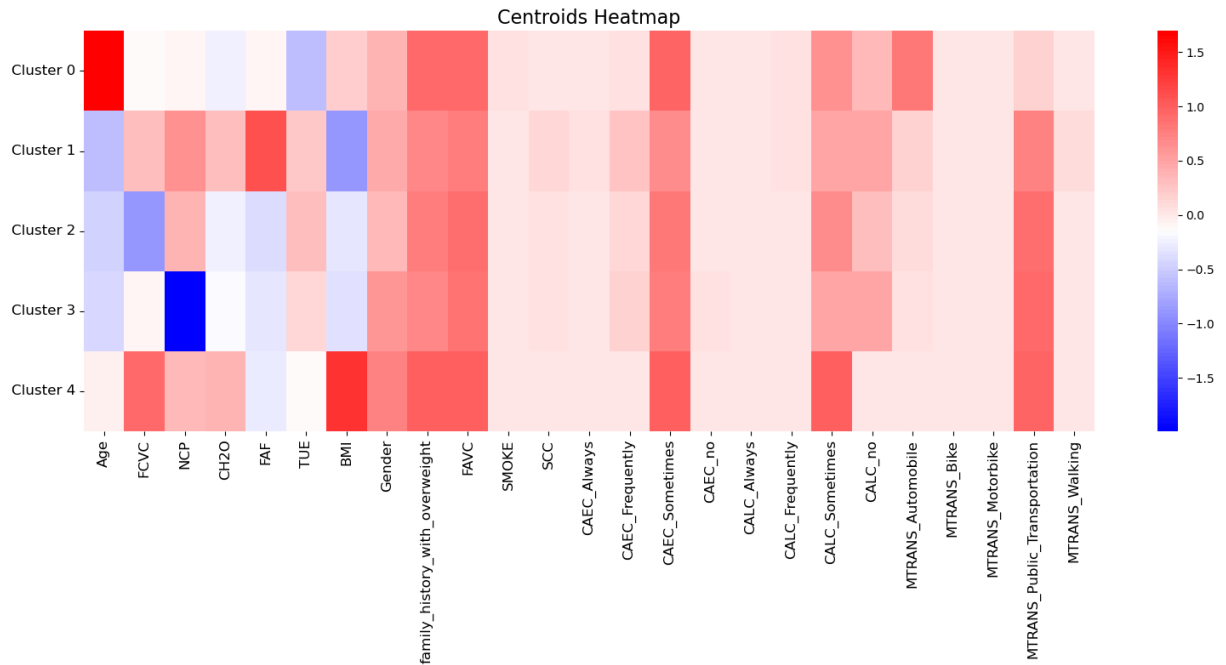
# Concatenar todos os nomes
feature_names = list(num_features) + list(bin_features) + list(cat_features)

len(feature_names), feature_names[:10]

centroids = kmeans.cluster_centers_

plt.figure(figsize=(16, 8))
sns.heatmap(
    centroids,
    cmap="bwr",
    annot=False,
    xticklabels=feature_names, # nomes reais
    yticklabels=[f"Cluster {i}" for i in range(kmeans.n_clusters)]
)

plt.title("Centroids Heatmap", fontsize=16)
plt.xticks(rotation=90, fontsize=12)
plt.yticks(rotation=0, fontsize=12)
plt.tight_layout()
plt.show()
```



Valor η^2 Classificação Interpretação para o Negócio ----- 0.01
 Pequeno A variável influencia muito pouco na definição dos grupos 0.06 Médio A variável tem influência moderada >0.14 Grande
 Variável Determinante. Fundamental para a segmentação- Diferente do Cohen's d (que compara pares), o η^2 mede o quanto a
 variável explica a variância global dos clusters.

```
In [36]: # df_clusters: dataframe contendo as variáveis e a coluna "cluster"
# cols_num: lista de variáveis numéricas
# Cohen's d funciona apenas para k = 2
eta_values = {}

for col in cols_num:
    grupos = [df_clusters[df_clusters["cluster"] == k][col].values for k in range(5)]

    # ANOVA F-statistic
    F, p = f_oneway(*grupos)

    # Cálculo de  $\eta^2$ : proporção da variância explicada pelo cluster
    grand_mean = df_clusters[col].mean()
    ss_between = sum([len(g) * (g.mean() - grand_mean)**2 for g in grupos])
    ss_total = sum((df_clusters[col] - grand_mean)**2)

    eta_squared = ss_between / ss_total
    eta_values[col] = eta_squared

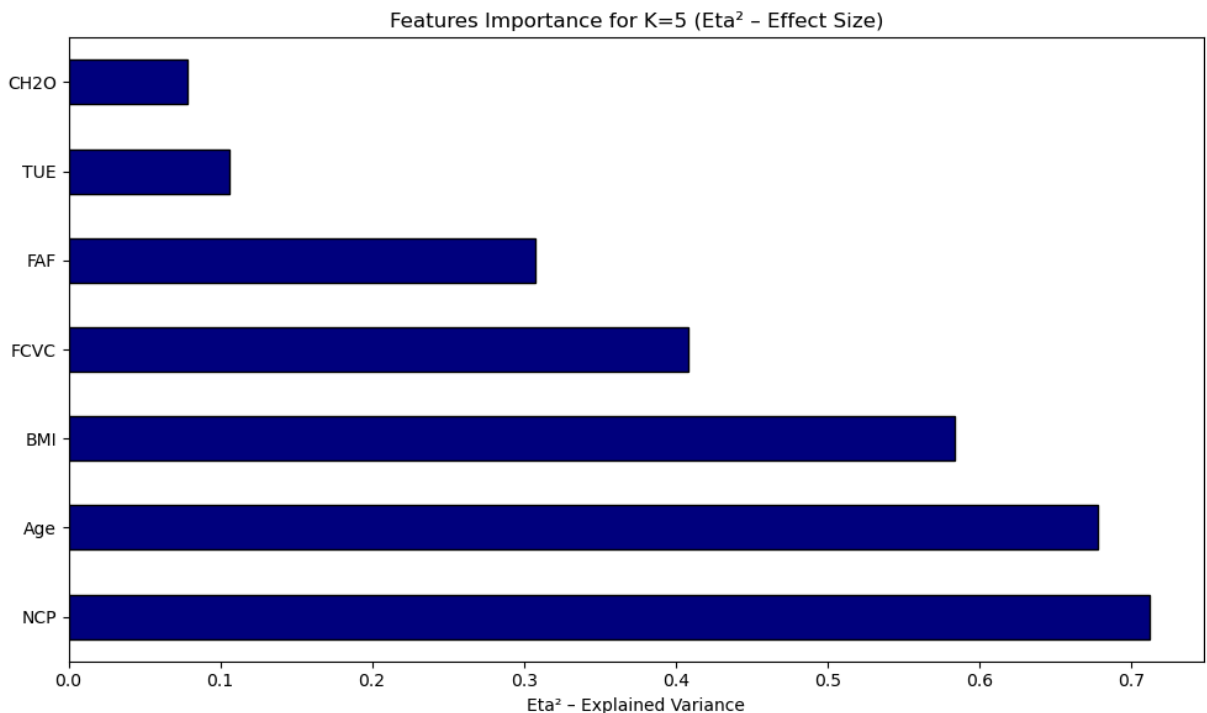
# Ranking
ranking_eta = pd.Series(eta_values).sort_values(ascending=False)

print("=== Ranking of the Most Determining Characteristics ( $\eta^2$  - ANOVA Effect Size) ===")
display(ranking_eta.to_frame("Eta^2"))

# Plot
ranking_eta.plot(kind="barh", figsize=(10,6), color="navy", edgecolor="black")
plt.title("Features Importance for K=5 ( $\eta^2$  - Effect Size)")
plt.xlabel(" $\eta^2$  - Explained Variance")
plt.tight_layout()
plt.show()
```


=== Ranking of the Most Determining Characteristics (Eta² - ANOVA Effect Size) ===

Eta ²	
NCP	0.712486
Age	0.678492
BMI	0.583539
FCVC	0.408325
FAF	0.307717
TUE	0.105949
CH2O	0.078177



Visualizações Avançadas para Validar Separabilidade

- Para corroborar a segmentação obtida pelo K-Means, utilizamos técnicas de redução de dimensionalidade linear (PCA) e não-linear (t-SNE e UMAP). Enquanto o t-SNE prioriza a estrutura local (vizinhança), o UMAP equilibra a preservação das distâncias globais, oferecendo duas perspectivas complementares sobre a separabilidade dos perfis. Adicionalmente, o dendrograma serve para explicar a anatomia e a história dessa separação.

- PCA

- Observações sobre o PCA: . Standard PCA (SVD): É o cálculo exato, perfeito e determinístico. O "padrão ouro"; . Incremental PCA: Serve para quando os dados não cabem na memória RAM; . Randomized PCA: Serve para quando o dataset é gigantesco; . Kernel PCA (kPCA) é não-linear, mas u-map e t-sne são muito superiores em criar visualizações 2D nítidas para clusters complexos. - A análise PCA (2D e 3D), combinada com as métricas internas, sugere: . Os clusters existem, mas não são fortemente definidos; . As fronteiras são suaves, contínuas e parcialmente sobrepostas; . K-Means consegue identificar “grupos de tendência”, não grupos isolados.

```

In [37]: # ---- PCA 2 componentes ----
pca2 = PCA(n_components=2, random_state=42)
X_pca2 = pca2.fit_transform(X_prep)

# ---- PCA 3 componentes ----
pca3 = PCA(n_components=3, random_state=42)
X_pca3 = pca3.fit_transform(X_prep)

# ---- Criação da figura ----
fig = plt.figure(figsize=(16, 6))

unique_clusters = np.unique(clusters)

# Paleta de cores consistente
cmap = plt.cm.get_cmap("tab10", len(unique_clusters))

# PCA 2D
ax1 = fig.add_subplot(1, 2, 1)

for cluster in unique_clusters:
    mask = clusters == cluster
    ax1.scatter(
        X_pca2[mask, 0], X_pca2[mask, 1],
        label=f"Cluster {cluster}",
        color=cmap(cluster),
        s=30, alpha=0.7
    )

ax1.set_title("PCA 2D - Clusters (k=5)")
ax1.set_xlabel("PC1")
ax1.set_ylabel("PC2")
ax1.grid(True)

# PCA 3D
ax2 = fig.add_subplot(1, 2, 2, projection='3d')

for cluster in unique_clusters:
    mask = clusters == cluster
    ax2.scatter(
        X_pca3[mask, 0], X_pca3[mask, 1], X_pca3[mask, 2],
        label=f"Cluster {cluster}",
        color=cmap(cluster),
        s=30, alpha=0.7
    )

ax2.set_title("PCA 3D - Clusters (k=5)")
ax2.set_xlabel("PC1")
ax2.set_ylabel("PC2")
ax2.set_zlabel("PC3")

# Legenda única (fora do gráfico)
handles, labels = ax1.get_legend_handles_labels()
fig.legend(
    handles, labels,
    loc="upper center",

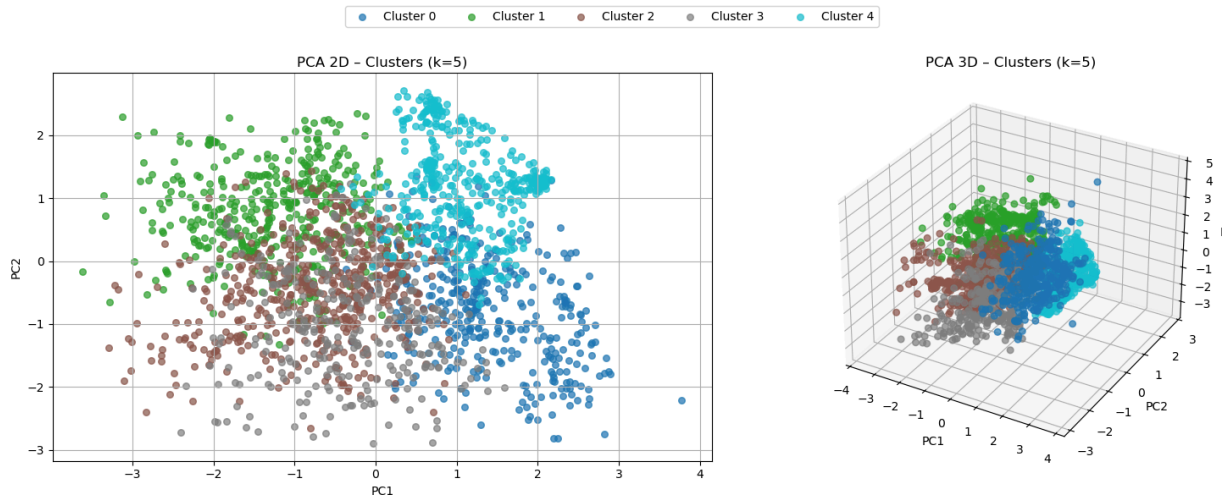
```

```

ncol=len(unique_clusters),
frameon=True,
fontsize=10
)

plt.tight_layout(rect=[0, 0, 1, 0.92]) # espaço para a legenda
plt.show()

```



- t-SNE e UMAP

- A análise do t-SNE e UMAP, sugere que: . A Ilha de "Alto Risco" (Cluster 4 - Roxo): Ambas as projeções confirmam que o Cluster 4 é o perfil mais discrepante da base de dados. Especialmente no UMAP, este grupo aparece em isolamento global extremo (canto inferior esquerdo), evidenciando que a Obesidade Mórbida (Tipo III) possui uma assinatura fenotípica radicalmente distante dos demais grupos, e não apenas uma "continuação" do sobrepeso; . O Gradiente de Transição (Clusters 0, 2 e 3): O UMAP revela que os Clusters 0 (Sênior), 2 (Jovem/Dieta Pobre) e 3 (Meal Skippers) formam um "continente" conectado. Isso sugere que, biologicamente, existe uma fronteira fluida entre o peso normal, o sobrepeso. A separação destes grupos se dá por nuances comportamentais específicas (ex: frequência de refeições), e não por uma barreira metabólica rígida; . A Coesão do Perfil "Fitness" (Cluster 1 - Laranja) Visualmente, o Cluster 1 destaca-se pela alta densidade e coesão. É uma "ilha" compacta e bem definida, indicando que o conjunto de hábitos saudáveis (alta atividade física + boa dieta) cria um padrão de dados muito homogêneo e de baixa variância, facilmente distinguível dos demais.

```

In [38]: # T-SNE
tsne_params = {
    "perplexity": [10, 20, 30, 40, 50],
    "learning_rate": [50, 100, 200, 300]
}

results_tsne = []

for p in tsne_params["perplexity"]:
    for lr in tsne_params["learning_rate"]:
        tsne = TSNE(
            n_components=2,
            perplexity=p,
            learning_rate=lr,
            max_iter=1000,
            random_state=42
        )

        embedding = tsne.fit_transform(X_prep)

```

```

        sil = silhouette_score(embedding, clusters)

        results_tsne.append({
            "perplexity": p,
            "learning_rate": lr,
            "silhouette": sil
        })

df_tsne = (
    pd.DataFrame(results_tsne)
    .sort_values("silhouette", ascending=False)
)

df_tsne.head()

```

Out[38]:

	perplexity	learning_rate	silhouette
18	50	200	0.339567
17	50	100	0.339132
19	50	300	0.336389
16	50	50	0.336307
15	40	300	0.320462

```

In [39]: umap_params = {
    "n_neighbors": [10, 20, 30, 50, 100],
    "min_dist": [0.0, 0.1, 0.3],
}

results_umap = []

for nn in umap_params["n_neighbors"]:
    for md in umap_params["min_dist"]:
        reducer = umap.UMAP(
            n_neighbors=nn,
            min_dist=md,
            random_state=42
        )
        embedding = reducer.fit_transform(X_prep)
        sil = silhouette_score(embedding, clusters)

        results_umap.append({
            "n_neighbors": nn,
            "min_dist": md,
            "silhouette": sil
        })

df_umap = pd.DataFrame(results_umap).sort_values("silhouette", ascending=False)
df_umap.head()

```

Out[39]:

	n_neighbors	min_dist	silhouette
12	100	0.0	0.313116
9	50	0.0	0.303583
14	100	0.3	0.293663
6	30	0.0	0.289373
7	30	0.1	0.287243

In [40]:

```
# Melhor TSNE
best_tsne = TSNE(
    n_components=2,
    perplexity=50,
    learning_rate=200,
    init="pca",
    random_state=42
)

# Melhor UMAP
best_umap = umap.UMAP(
    n_components=2,
    n_neighbors=100,
    min_dist=0.0,
    metric="euclidean",
    random_state=42
)

# Embeddings
X_tsne = best_tsne.fit_transform(X_prep)
X_umap = best_umap.fit_transform(X_prep)

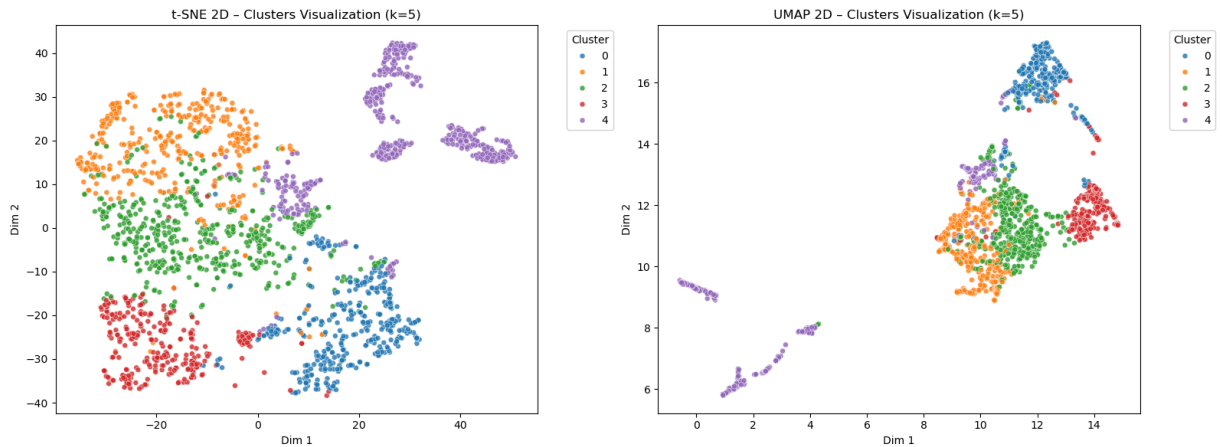
# Subplots
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# t-SNE Plot
sns.scatterplot(
    x=X_tsne[:, 0], y=X_tsne[:, 1],
    hue=clusters, palette="tab10",
    s=25, alpha=0.8, ax=axes[0]
)
axes[0].set_title("t-SNE 2D - Clusters Visualization (k=5)")
axes[0].set_xlabel("Dim 1")
axes[0].set_ylabel("Dim 2")
axes[0].legend(title="Cluster", bbox_to_anchor=(1.05, 1), loc="upper left")

# UMAP Plot
sns.scatterplot(
    x=X_umap[:, 0], y=X_umap[:, 1],
    hue=clusters, palette="tab10",
    s=25, alpha=0.8, ax=axes[1]
)
axes[1].set_title("UMAP 2D - Clusters Visualization (k=5)")
axes[1].set_xlabel("Dim 1")
```

```
axes[1].set_ylabel("Dim 2")
axes[1].legend(title="Cluster", bbox_to_anchor=(1.05, 1), loc="upper left")

plt.tight_layout()
plt.show()
```



- Dendrograma

- Ao traçar a linha em 37, podemos contar exatamente 5 intersecções com as linhas verticais (os ramos do dendrograma): . O ramo laranja (perfil 1); . A primeira subdivisão do grande ramo verde (perfil 2); . A segunda subdivisão do ramo verde (perfil 3); . O ramo vermelho (perfil 4); . O ramo roxo (perfil 5).

```
In [43]: # Dendrograma
# Gerar o Linkage
linkage_matrix = sch.linkage(X_prep, method='ward')

# Mostrar as alturas das últimas 10 fusões
# As colunas são: [idx1, idx2, dist, sample_count]
last_merges = linkage_matrix[-10:]

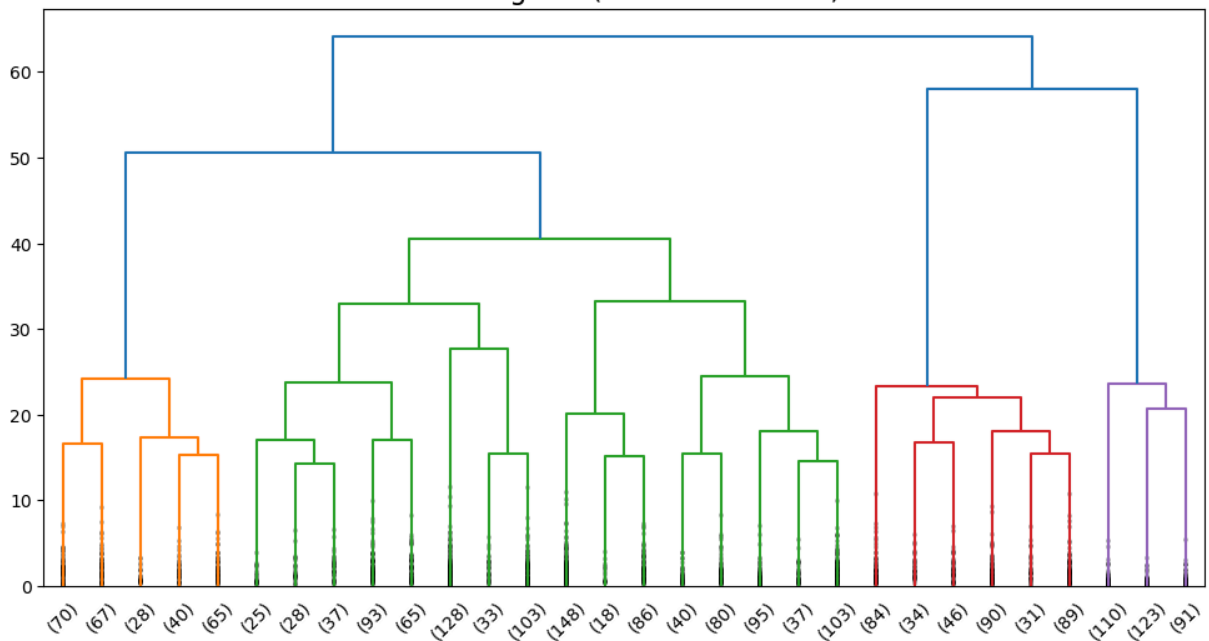
print("--- Heights of the last 10 mergers (Ward's Distance) ---")
for i, merge in enumerate(last_merges):
    print(f"Fusão {i+1} (Passo {len(linkage_matrix)-10+i}): Altura = {merge[2]:.2f}")

# Plotar o Dendrograma sem linha de corte primeiro
plt.figure(figsize=(12, 6))
plt.title("Dendrogram (Check the Y-axis)", fontsize=16)
dendrogram = sch.dendrogram(
    linkage_matrix,
    truncate_mode='lastp',
    p=30,
    leaf_rotation=45.,
    show_contracted=True
)
plt.show()
```

--- Heights of the last 10 mergers (Ward's Distance) ---

Fusão 1 (Passo 2076): Altura = 23.74
Fusão 2 (Passo 2077): Altura = 24.14
Fusão 3 (Passo 2078): Altura = 24.42
Fusão 4 (Passo 2079): Altura = 27.73
Fusão 5 (Passo 2080): Altura = 32.94
Fusão 6 (Passo 2081): Altura = 33.17
Fusão 7 (Passo 2082): Altura = 40.50
Fusão 8 (Passo 2083): Altura = 50.57
Fusão 9 (Passo 2084): Altura = 58.06
Fusão 10 (Passo 2085): Altura = 64.16

Dendrogram (Check the Y-axis)



```
In [45]: # Configuração da figura
plt.figure(figsize=(12, 6))

# Título e Labels
plt.title("Hierarchical Dendrogram: Structure Validation (k=5)", fontsize=16)
plt.xlabel("Individuals", fontsize=12)
plt.ylabel("Ward's Distance (Dissimilarity)", fontsize=12)

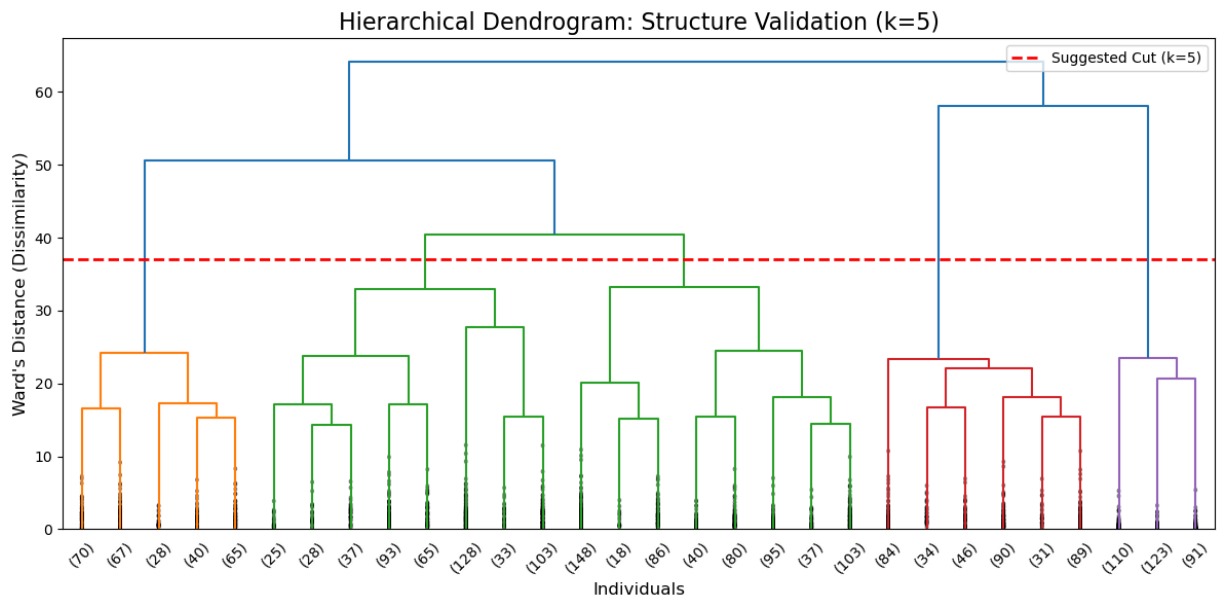
# Gerando o Linkage Matrix
linkage_matrix = sch.linkage(X_prep, method='ward')

dendrogram = sch.dendrogram(
    linkage_matrix,
    truncate_mode='lastp',
    p=30,
    show_leaf_counts=True,
    leaf_rotation=45.,
    leaf_font_size=10.,
    show_contracted=True,
)

# LINHA DE CORTE AJUSTADA
# A heurística de olhar as duas últimas fusões e fazer a média para fazer o corte
```

```
# não dividiria em 5 clusters,
# Neste caso, a média entre a fusão 6 e 7 é a ideal para o ponto de corte.
plt.axhline(y=37.0, color='r', linestyle='--', linewidth=2, label='Suggested Cut (k=5)')

plt.legend()
plt.tight_layout()
plt.show()
```



In [46]: `df_prep.head()`

Out[46]:

	Gender	Age	family_history_with_overweight	FAVC	FCVC	NCP	CAEC	SMOKE	C
0	1	21.0		1	0	2.0	3.0	Sometimes	0
1	1	21.0		1	0	3.0	3.0	Sometimes	1
2	0	23.0		1	0	2.0	3.0	Sometimes	0
3	0	27.0		0	0	3.0	3.0	Sometimes	0
4	0	22.0		0	0	2.0	1.0	Sometimes	0

Salvando o Pipeline

```
In [60]: # cols para o pré-processador
cols_num = ["Age", "FCVC", "NCP", "CH20", "FAF", "TUE", "BMI"]
cols_cat = ["CAEC", "CALC", "MTRANS"]
cols_bin = ["Gender", "family_history_with_overweight", "FAVC", "SMOKE", "SCC"]

# A Lista completa na ordem que o preprocessor foi treinado
features_set = cols_num + cols_bin + cols_cat
projeto_cancer = {
    'preprocessor': preprocessor,
    'kmeans': kmeans,
    'features': cols_num,
```



```

'clusters': clusters,
'X_pca2': X_pca2,
'X_pca3': X_pca3,
'X_tsne': X_tsne,
'X_umap': X_umap,
'linkage_matrix': linkage_matrix,
'threshold_dendro': 37.0,
'clusters_desc': {
    0: ("👴 Sênior Sedentário", "Indivíduos mais velhos que dependem de automóv
    1: ("🏃 Fitness / Saudável", "Grupo jovem com alta atividade física e diet
    2: ("🎓 Jovem / Dieta Pobre", "Comum entre estudantes. Baixo consumo de ve
    3: ("🍷 Meal Skipper", "Caracterizado pela rotina alimentar instável e o h
    4: ("🚨 Alto Risco (Obesidade III)", "Estado metabólico isolado com altíss
}

}

joblib.dump(projeto_obesidade, 'modelo_cluster_obesidade_vtl.pkl')

```

Out[60]: ['modelo_cluster_obesidade_vtl.pkl']

Síntese

- Objetivo: O projeto teve como objetivo identificar agrupamentos naturais (clusters) de indivíduos no dataset de obesidade da UCI, mapeando padrões latentes entre hábitos alimentares, condições físicas e variáveis demográficas. O foco foi validar se os níveis de obesidade possuem fronteiras biológicas/comportamentais rígidas ou se comportam como um espectro contínuo.

- Metodologia resumida: . Análise Exploratória (EDA): Avaliação de distribuições, correlações numéricas e associações categóricas via V de Cramér Corrigido; . Pré-processamento: Pipeline unificado utilizando OneHotEncoder para categóricas e StandardScaler para numéricas, preservando a variância dos outliers para a clusterização; . Modelagem Comparativa: Teste exaustivo de algoritmos (K-Means, GMM, Agglomerative Clustering, BIRCH e K-Means++) avaliados por métricas internas (Silhouette, Davies-Bouldin, Calinski-Harabasz); . Validação Visual Avançada: Uso de reduções de dimensionalidade linear (PCA) e não-linear (t-SNE, UMAP) e dendrograma para corroborar a topologia dos clusters encontrados (5).

- Resultados Principais: . Modelo Vencedor: O algoritmo K-Means (k=5) apresentou o melhor equilíbrio entre coesão e interpretabilidade de negócio. . Perfis Identificados: * Cluster 0 (Sênior): Indivíduos mais velhos (média 35 anos), sedentários e dependentes de automóvel; * Cluster 1 (Fitness/Saudável): Grupo jovem, com alta atividade física, alta ingestão de vegetais e água. O perfil mais coeso e "exemplar"; * Cluster 2 (Jovem/Dieta Pobre): Estudantes/Jovens que utilizam transporte público e possuem baixo consumo de vegetais; * Cluster 3 (Meal Skippers): Caracterizados por fazerem poucas refeições ao dia (NCP approx 1.5), indicando rotina desregulada; * Cluster 4 (Alto Risco/Obesidade III): O grupo mais extremo, isolado geometricamente, com alto IMC, histórico familiar de obesidade e alto consumo calórico, apesar de relatar ingestão de vegetais.

- Conclusões: . A análise revelou uma estrutura dual nos dados: * Discrepância da Obesidade Mórbida: As projeções UMAP confirmaram que a Obesidade Tipo III (Cluster 4) não é apenas uma continuação do sobrepeso, mas um estado metabólico e comportamental isolado (outlier global); * Espectro Contínuo: Os demais perfis (Peso Normal a Sobrepeso) formam um "continente" conectado, sugerindo que a transição é gradual e influenciada por nuances comportamentais (como pular refeições ou sedentarismo) e não por barreiras rígidas; * Coesão do Hábito Saudável: O estilo de vida saudável (Cluster 1) cria um padrão de dados muito denso e distinto, fácil de ser segregado dos demais.

- Limitações: . Linearidade: O PCA (linear) falhou em separar visualmente os grupos, evidenciando a complexidade não-linear das relações biológicas; . Sobreposição: A existência de fronteiras suaves entre os clusters de "meio de espectro" (Clusters 0, 2 e 3) impede uma separação perfeita, refletindo a natureza contínua do IMC.

- Sugestões para Trabalhos: . Futuros Modelagem Preditiva: Utilizar os labels gerados pelo K-Means (1-5) como target para treinar um classificador (ex: Random Forest ou XGBoost) e identificar as features mais importantes via SHAP Values; . Feature Engineering: Criar novas variáveis de interação (ex: Calorias/Atividade) para tentar aumentar a distância entre os clusters sobrepostos (0, 2 e 3); . Análise Temporal: Se disponível, coletar dados longitudinais para entender a migração de indivíduos entre os clusters (ex: como um indivíduo do Cluster 2 "migra" para o Cluster 4 ao longo do tempo).